

USART Driver Analysis – STM32F446RE

Introduction

The USART driver is developed to simplify serial communication using the USART peripherals available in the STM32F446RE microcontroller. Instead of directly accessing USART registers in the application code, the driver provides a set of APIs that handle all required register-level operations internally. This approach improves code readability, reusability, and maintainability while reducing the complexity of peripheral configuration.

How the Driver Works

The USART driver acts as an abstraction layer between the user application and the USART hardware of the STM32F446RE microcontroller. When the application needs to perform serial communication, it interacts with the USART driver APIs rather than directly modifying hardware registers. During USART initialization, the application first fills the USART handle structure with the required communication parameters such as USART mode, baud rate, stop bits, word length, parity settings, and hardware flow control options. The `USART_Init ()` API then reads these parameters and configures the corresponding USART registers automatically. This design allows different USART peripherals to be configured using the same API without modifying the driver source code. As a result, the driver hides low-level register operations from the application and provides a simple and reusable interface for serial communication.

For example:

- Baud rate configuration → BRR register
- Word length configuration → CR1 register
- Parity control configuration → CR1 register
- Transmitter and Receiver enable → CR1 register
- Stop bit configuration → CR2 register
- Hardware flow control configuration → CR3 register
- Data transmission and reception → DR register
- Status monitoring → SR register

The application does not need to know which register or bit position must be modified because the driver handles all register-level operations internally.

Hardware Abstraction

Without a driver, configuring a USART peripheral requires direct register manipulation.

Example:

```
RCC->APB1ENR |= (1<<17);
```

```
USART2->CR1 |= (1<<13);
```

With the USART driver:

```
USART_Handle_t USART2Handle;
```

```
USART2Handle.pUSARTx = USART2;
```

```
USART2Handle.USART_Config.USART_Mode = USART_MODE_TXRX;
```

```
USART2Handle.USART_Config.USART_Baud = USART_STD_BAUD_115200;
```

```
USART2Handle.USART_Config.USART_NoOfStopBits = USART_STOPBITS_1;
```

```
USART2Handle.USART_Config.USART_WordLength = USART_WORDLEN_8BITS;
```

```
USART2Handle.USART_Config.USART_ParityControl = USART_PARITY_DISABLE;
```

```
USART2Handle.USART_Config.USART_HWFlowControl =  
USART_HW_FLOW_CTRL_NONE;
```

```
USART_Init (&USART2Handle);
```

This approach hides low-level register manipulation from the application developer and provides a simple API-based interface for USART configuration and communication.

USART Driver APIs

The USART_PeriClockControl () API is used to enable or disable the peripheral clock for a USART module. Before configuring any USART peripheral, the corresponding peripheral clock must be enabled. This API ensures that the USART peripheral receives the required clock from the RCC module.

The USART_Init () API is responsible for configuring a USART peripheral according to the parameters specified in the USART handle structure. It configures communication mode, baud rate, stop bits, word length, parity settings, and hardware flow control options. This is the primary API used during USART initialization.

The `USART_DeInit ()` API resets the selected USART peripheral back to its default reset state. This can be useful when reconfiguring the peripheral or restoring default settings.

The `USART_SendData ()` API is used to transmit data using polling-based communication. The API waits until the transmit data register becomes empty and then loads the next data byte for transmission.

The `USART_ReceiveData ()` API is used to receive data using polling-based communication. The API continuously monitors the receive status flags and retrieves incoming data from the USART peripheral.

The `USART_SendDataIT ()` API provides interrupt-based data transmission. Instead of continuously polling status flags, the driver uses interrupts to transmit data efficiently while allowing the CPU to perform other tasks.

The `USART_ReceiveDataIT ()` API provides interrupt-based data reception. Incoming data is received through interrupt events, improving system responsiveness and reducing CPU utilization.

The `USART_IRQInterruptConfig ()` API is used to enable or disable USART interrupt requests within the NVIC controller.

The `USART_IRQPriorityConfig ()` API assigns interrupt priority levels to USART interrupt sources, allowing proper interrupt management when multiple interrupts occur simultaneously.

The `USART_IRQHandling()` API handles USART interrupt events and processes transmission, reception, and error-related interrupt conditions.

The `USART_GetFlagStatus()` API is used to check the status of USART flags such as transmit buffer empty, receive buffer not empty, and transmission complete conditions.

The `USART_ClearFlag ()` API clears status flags whenever required by the application.

The `USART_PeripheralControl()` API enables or disables the USART peripheral after configuration.

The `USART_SetBaudRate ()` API calculates and configures the baud rate value based on the peripheral clock frequency and the user-selected baud rate.

The `USART_ApplicationEventCallback()` function allows the application layer to receive notification events such as transmission completion, reception completion, idle detection, parity errors, framing errors, noise errors, and overrun errors.

Practical Example Used in This Project

To verify the functionality of the USART driver, USART2 was configured for serial communication using the driver APIs. The application initialized the USART peripheral with a baud rate of 115200 bps and enabled both transmission and reception. Test messages were transmitted through the USART peripheral and successfully received on the connected terminal interface. Successful communication confirmed that the baud rate configuration, transmit logic, receive logic, interrupt handling mechanisms, and register configurations implemented in the driver were functioning correctly.

Conclusion

The USART driver developed for the STM32F446RE microcontroller provides a structured and reusable interface for serial communication. The driver successfully abstracts low-level hardware operations and allows the application to interact with USART peripherals through a simple set of APIs. By hiding register-level complexity and providing a modular software architecture, the USART driver simplifies embedded application development, improves code maintainability, and enables reliable serial communication for a wide range of embedded applications.